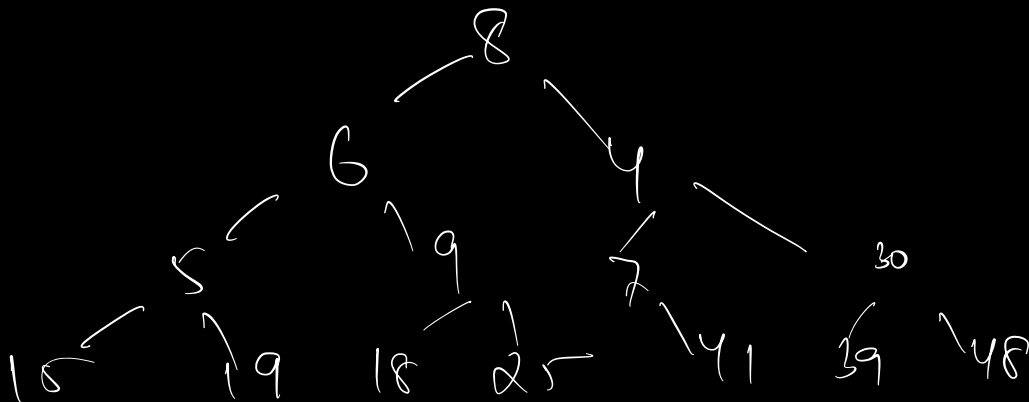
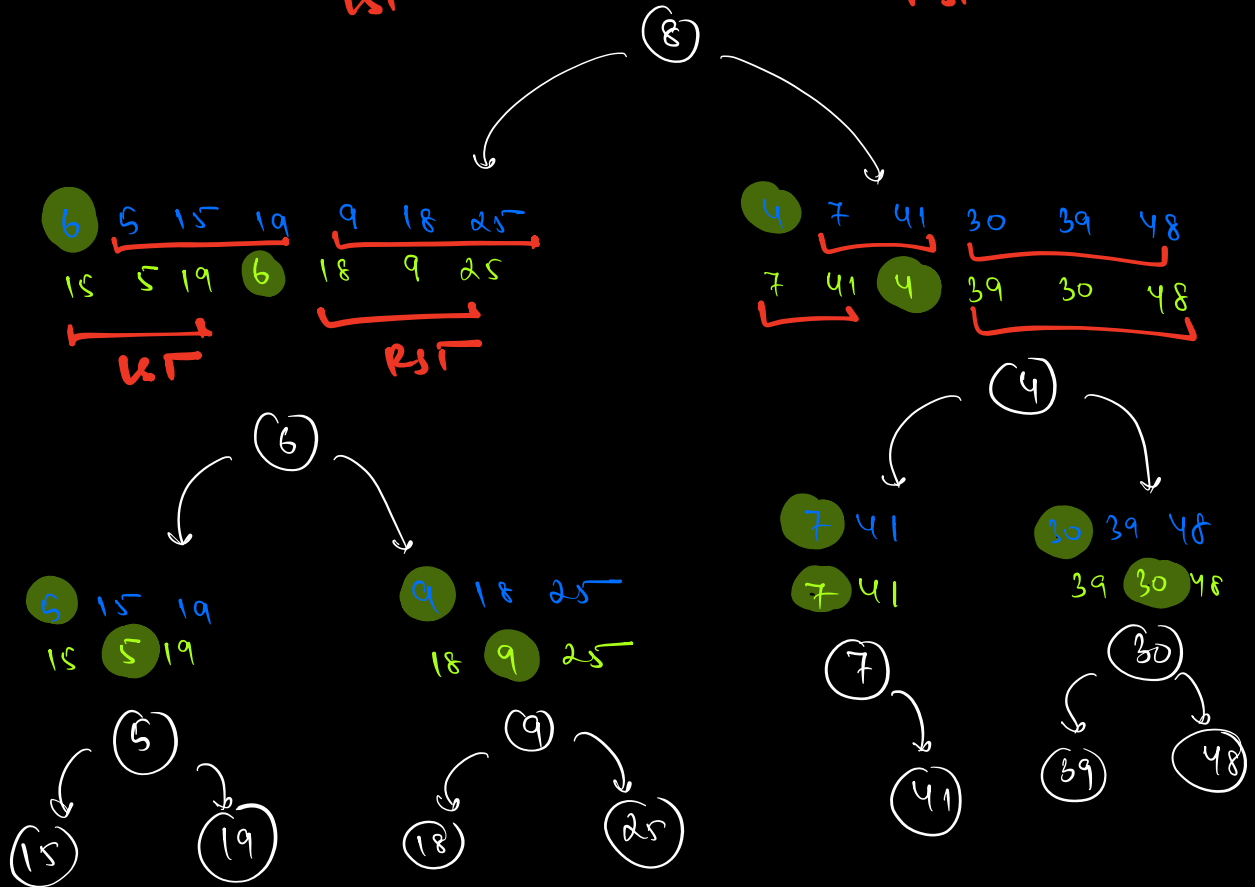


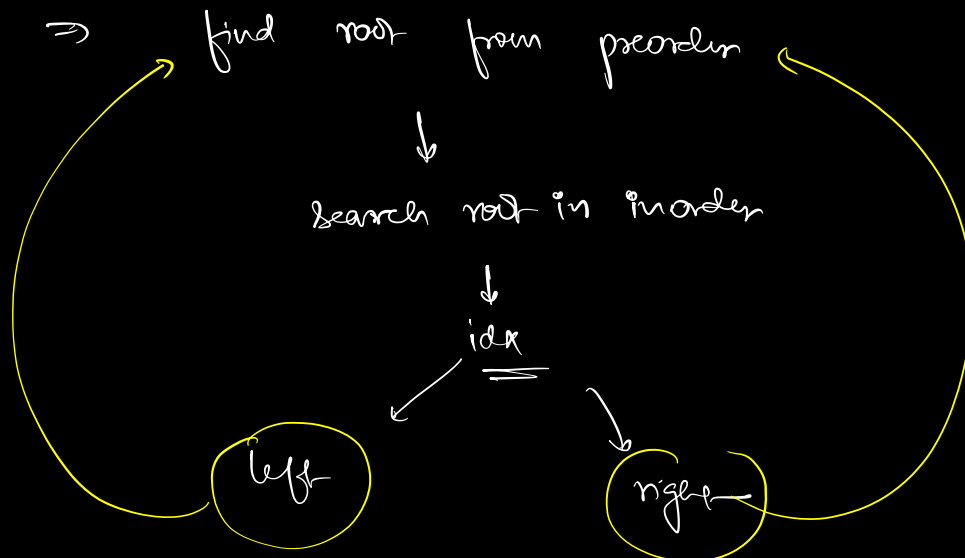
⇒ Construct a binary tree { distinct elements }

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
pre ⇒	8	6	5	15	19	9	18	25	4	7	41	30	39	48
in ⇒	15	5	19	6	18	9	25	8	7	41	4	39	30	48

LSI
RST



⇒ get the root from preorder



ins - inorder start -

inf - inorder end

ps - pre start -

pe - pre end

```
TreeNode construct (preTs, inTs, ps, pe, ins, inf) {
```

```
    if (ps > pe)
```

```
        return NULL;
```

```
    int rootElement = pre[ps];
```

```
    int rootId = -1;
```

```
    for (i = ins; i <= inf; i++) {
```

```
        if (inorder[i] == rootElement) {
```

```
            rootId = i;
```

```
            break;
```

```
        }
```

```
    }
```

Optimise
to $O(1)$
using hashmap
N

```

→ TreeNode n = new TreeNode(rootElement);
lengths[] = rootIdx - ins;

```

```

n.left = construct(pre, in, ps+1, ps+lengths[],
                  ins, rootIdx-1)

```

```

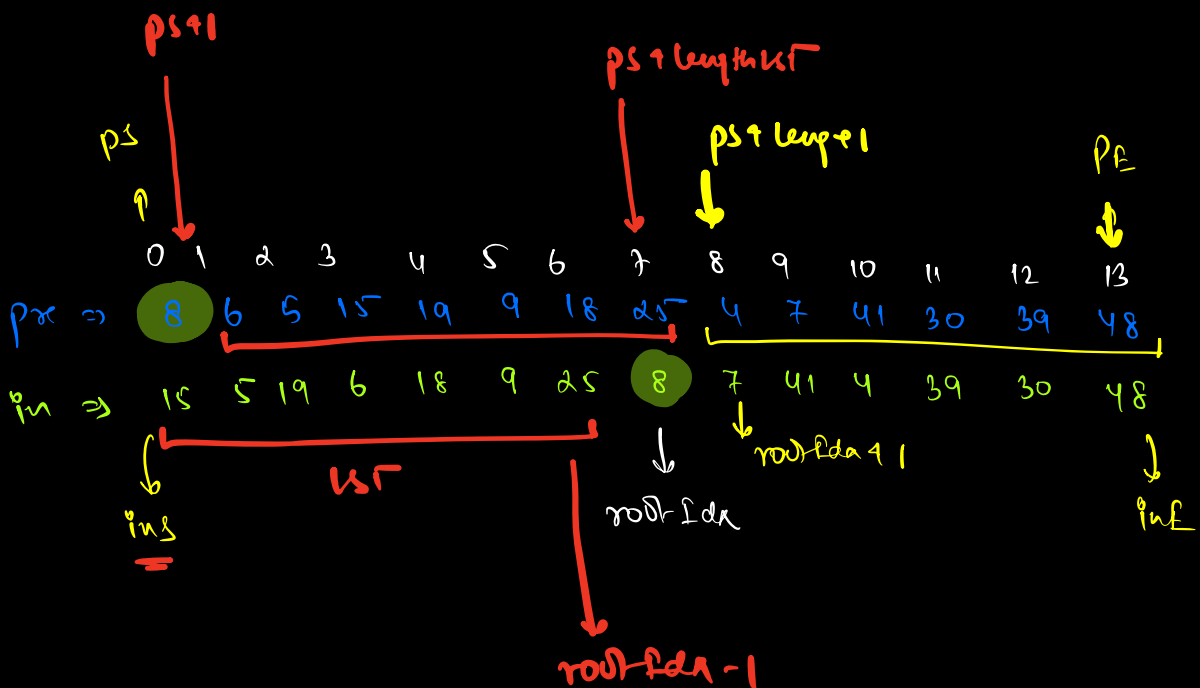
n.right = construct(pre, in, ps+lengths[]+1, pe, rootIdx+1,
                  inc)
return n;

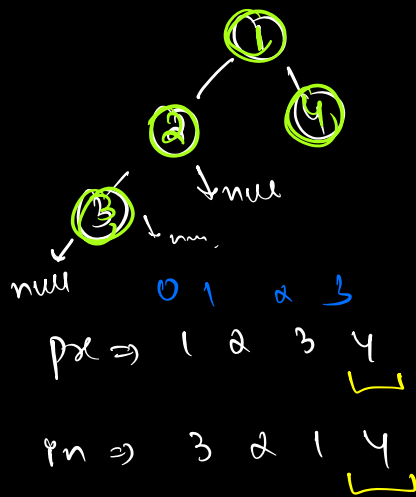
```

```

}

```





ps pc pns pnf

cons(3, 3, 3, 3)

cons(0, 3, 0, 3)

root = 4

root = 3

length = 3 - 3 = 0

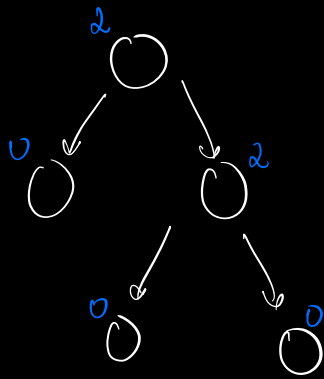
list

TC = $O(N)$

⇒ types of binary tree :-

1) Full Binary tree : has 0 or 2 children for each node

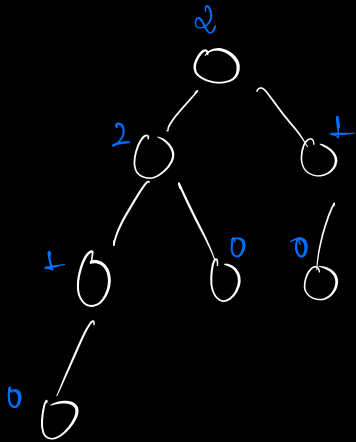
2) Complete binary tree : each and every should be complete except last level, should be filled L-R.



→

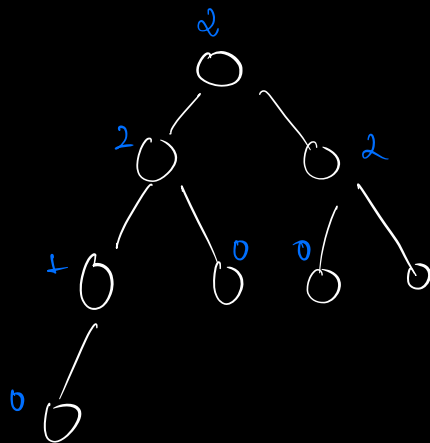
FBT ✓

CBT ✗



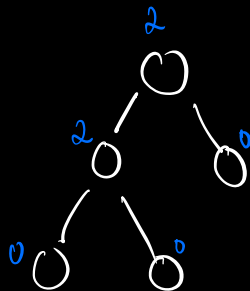
FBT — ✗

CBT — ✗



FBT → ✗

CBT → ✓



FBT — ✓

CBT — ✓

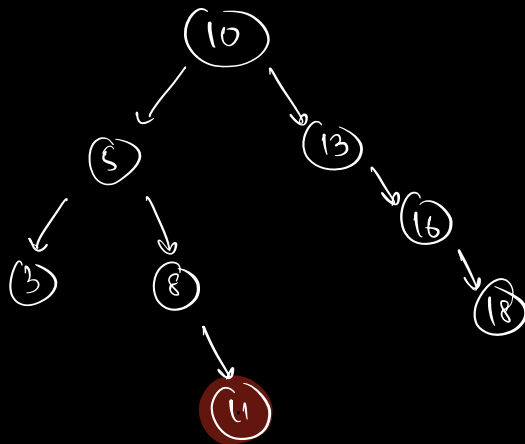
⇒ BINARY SEARCH TREES



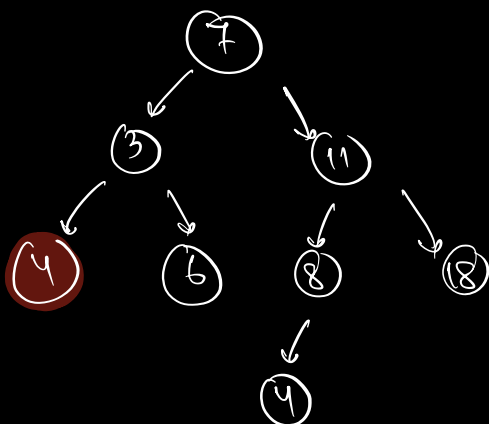
binary
trees → ①

$$\left(\begin{array}{c} \text{all nodes} \\ \text{LST} \end{array} \right) < \text{root} < \left(\begin{array}{c} \text{all nodes} \\ \text{RST} \end{array} \right)$$

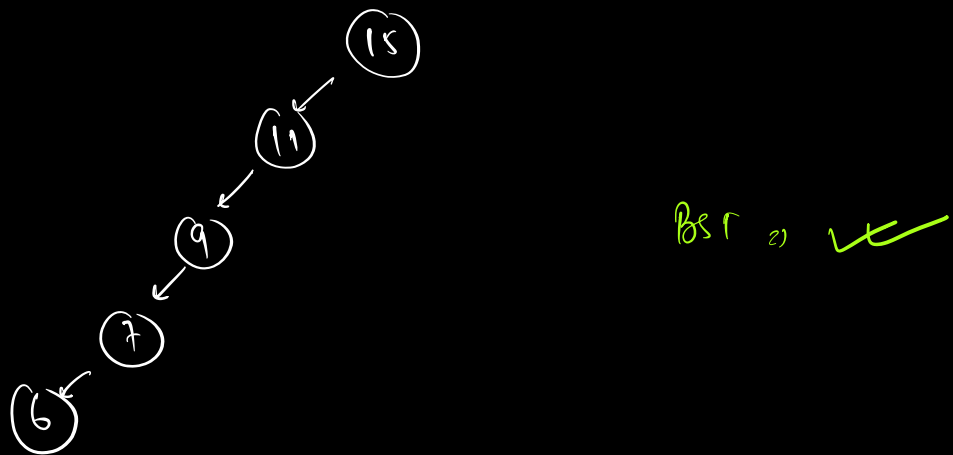
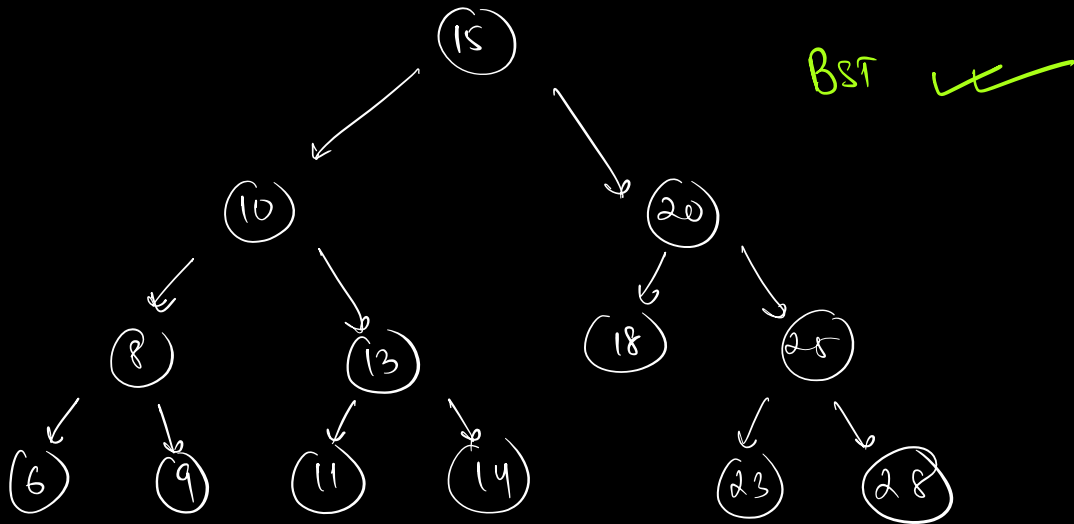
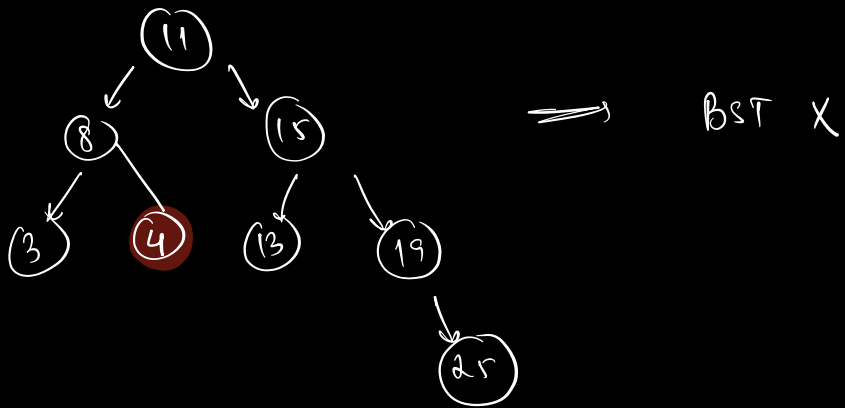
↓
for all nodes

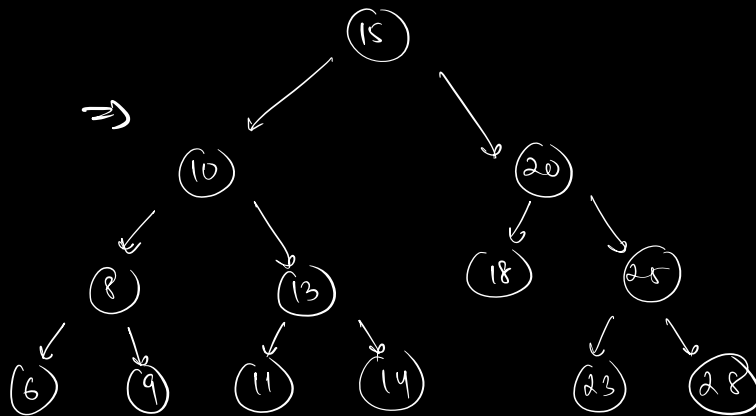


⇒ not a BST

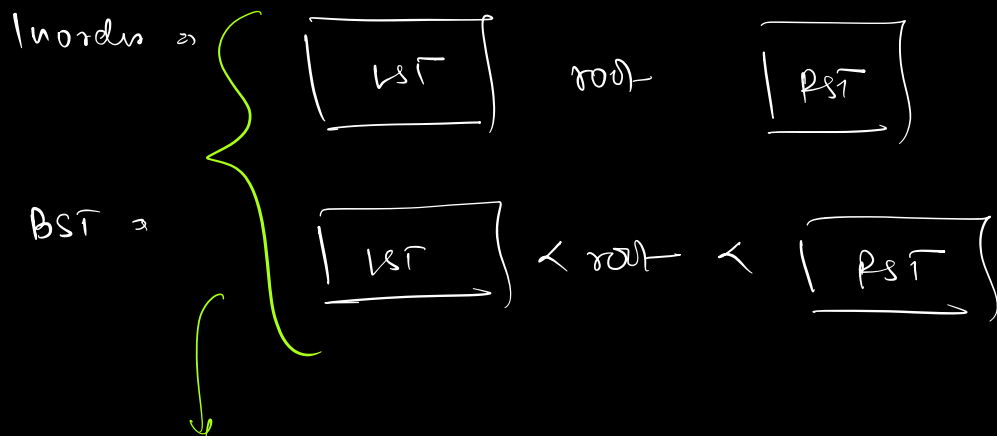


⇒ BST ⇒ X





⇒ Inorder traversal ⇒ 6 8 9 10 11 13 14 15 18 20 23 25 28

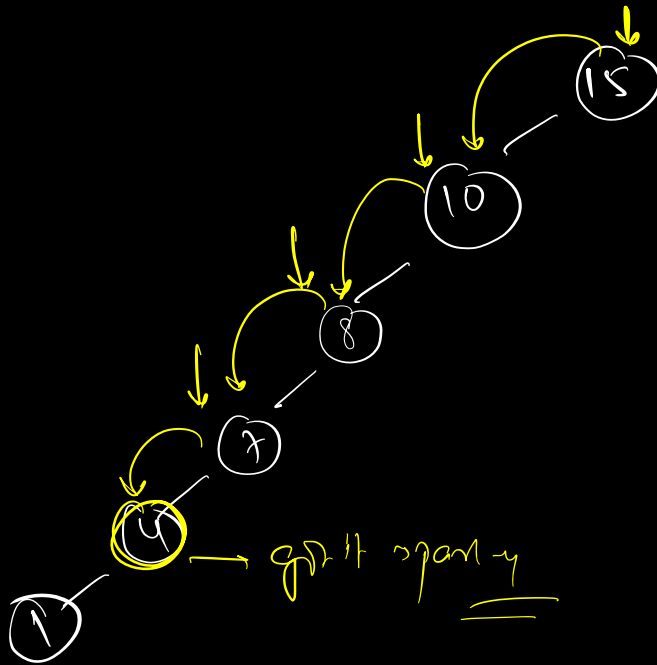
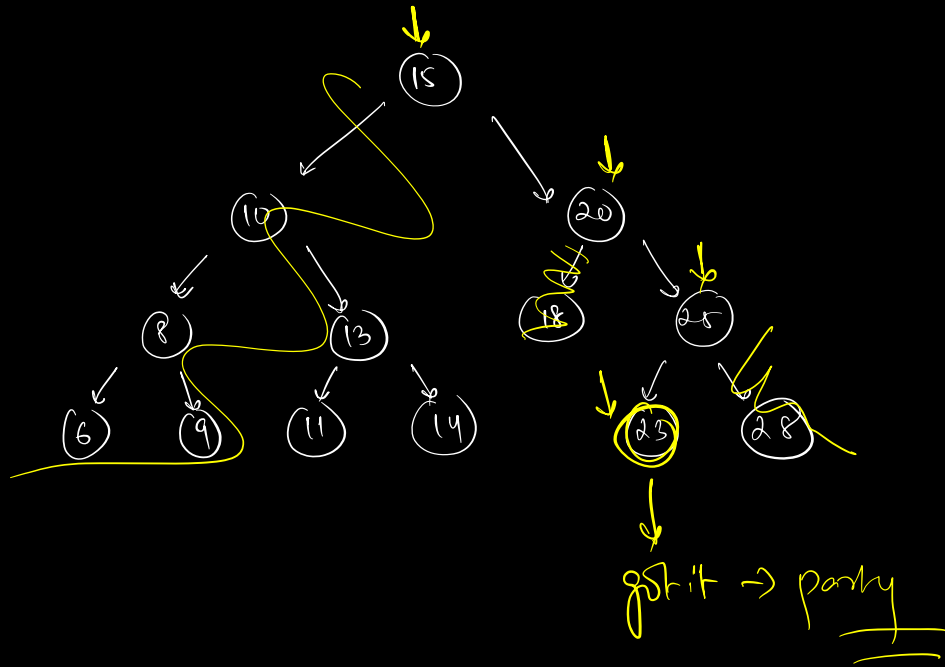


Inorder of a BST is always sorted.

BST ⇒ Binary search tree

searching

$k = 23$



$k = 4$

BSF $\rightarrow O(\log N) \rightarrow$ balanced BSF

skewed $\rightarrow O(N)$

BBST

(AVL & Red black)
trees trees

```

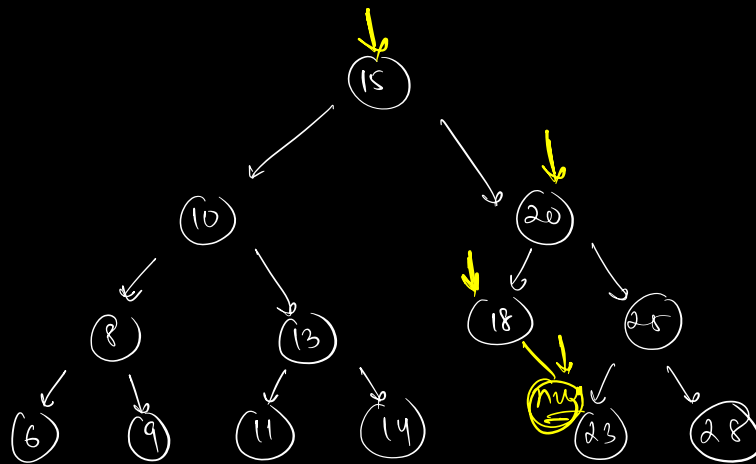
boolean search (TreeNode root, int k) {
    TreeNode temp = root;

    while (temp != null) {
        if (temp.data == k) {
            [let's party]
            return true;
        }
        else if (temp.data < k) {
            temp = temp.right;
        }
        else {
            temp = temp.left;
        }
    }
    return false;
}

```

Recursive Code → H.W

⇒ Insert a node in a BST:



19

```
boolean search (TreeNode root, int k) {
```

```
    TreeNode temp = root;
    parent = root;
```

```
    while (temp != null) {
```

```
        if (temp.data == k) {
```

```
            break;
```

```
        } else if (temp.data < k) {
```

```
            temp = temp.right;
```

```
        } else {
```

```
            temp = temp.left;
```

```
        }
```

```
    if (parent == null)
```

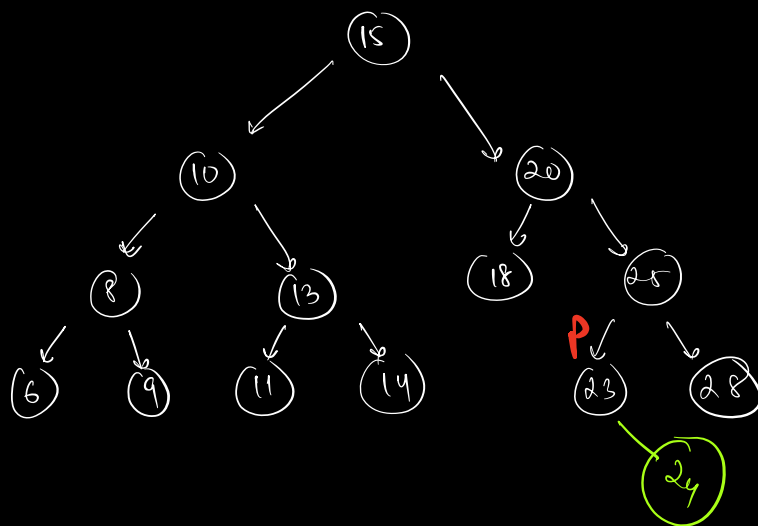
```
        return new Node(k);
```

```
}
```

```

if (parent.data > k) {
    parent.left = new Node(k)
}
if (parent.data < k) {
    parent.right = new Node(k)
}
}

```

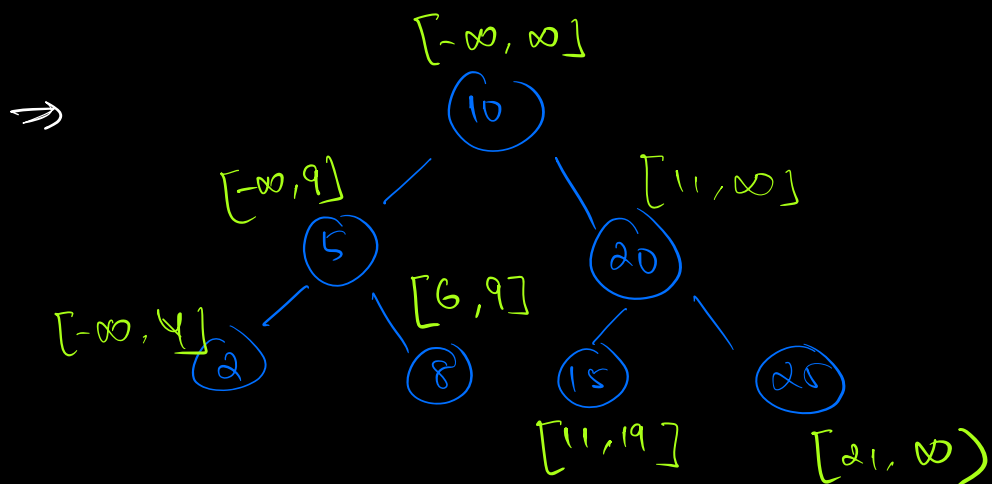


TC $\rightarrow$ same as searching

⇒ Check if a BT is a BST?

1) check if inorder is sorted

$$\begin{array}{l} TC \Rightarrow O(N) \\ SC = O(N) \end{array}$$



⇒ 8 Dec → HW

5 mins